



CIRCUITOS DIGITALES Y MICROCONTROLADORES 2026

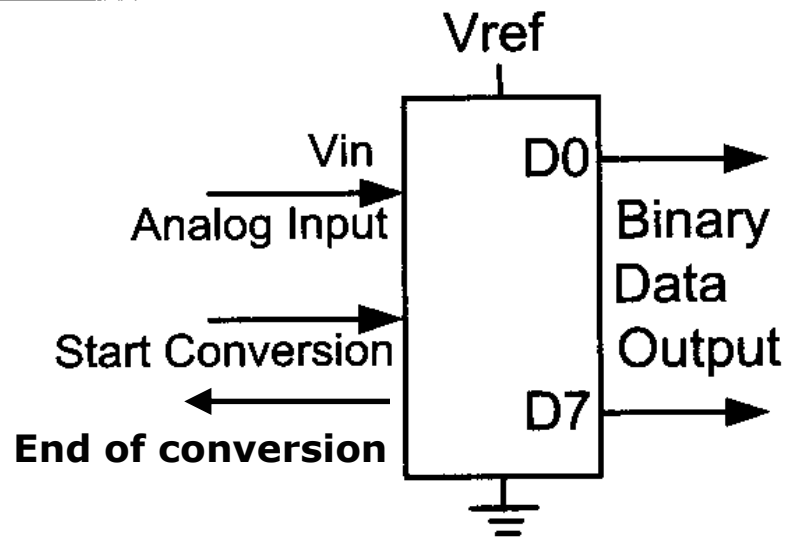
Facultad de Ingeniería
UNLP

PERIFERICOS
ANALÓGICOS

Ing. José Juárez

Introducción al periférico ADC

- Un **conversor Analógico - Digital (ADC en inglés)** permite convertir magnitudes analógicas a valores digitales (números binarios) que luego pueden ser procesados en una CPU (con un número finito de bits).
- Esquema general de un ADC:



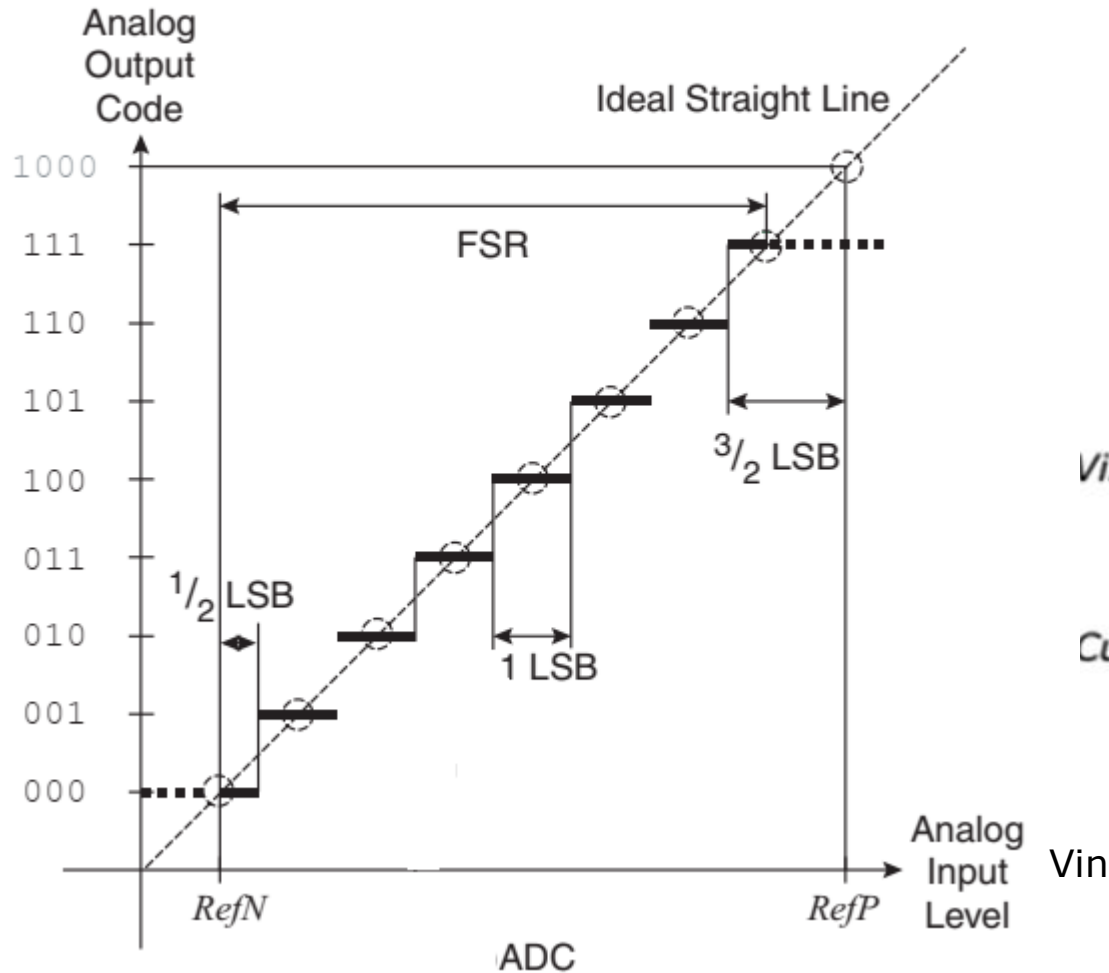
Introducción al periférico ADC

Algunas definiciones y parámetros a considerar:

- **Resolución o paso de cuantización:** es la mínima variación de la tensión de entrada que cambia el valor de salida en un LSB.
- **Precisión salida:** Es el número de bits del resultado de la conversión (8, 10, 12, 16 y hasta 24bits)
- **Tensión de Referencia:** (V_{ref}) es proporcionada externamente por un terminal de entrada. Debe ser estable ya que de ella depende la resolución.
- **Rango analógico de entrada:** rango de variación de la señal analógica de entrada, está fijado por la tensión de referencia (generalmente expresada como V_{refH} - V_{refL} o simplemente V_{ref})
- **Tiempo de conversión:** es el tiempo que tarda el ADC en entregar el resultado de una conversión. Se mide desde que se inicia la conversión hasta que se activa el flag de fin de conversión. Depende del reloj y del tipo de ADC
- **Máxima Tasa de conversión:** es la cantidad máxima de conversiones que el ADC puede realizar en un segundo. Se mide en SPS (samples per second). Depende del reloj y del tipo de ADC.

Introducción al periférico ADC

Ideal Transfer Functions



$$LSB_{ADC} = \frac{RefP - RefN}{2^N}$$

$$Output = ROUND((V_{in} - RefN) / LSB)$$

$$RefN \leq V_{in} < RefP - LSB$$

$$V_{in} \approx RefN + Output \cdot LSB \quad \text{Mientras que:}$$

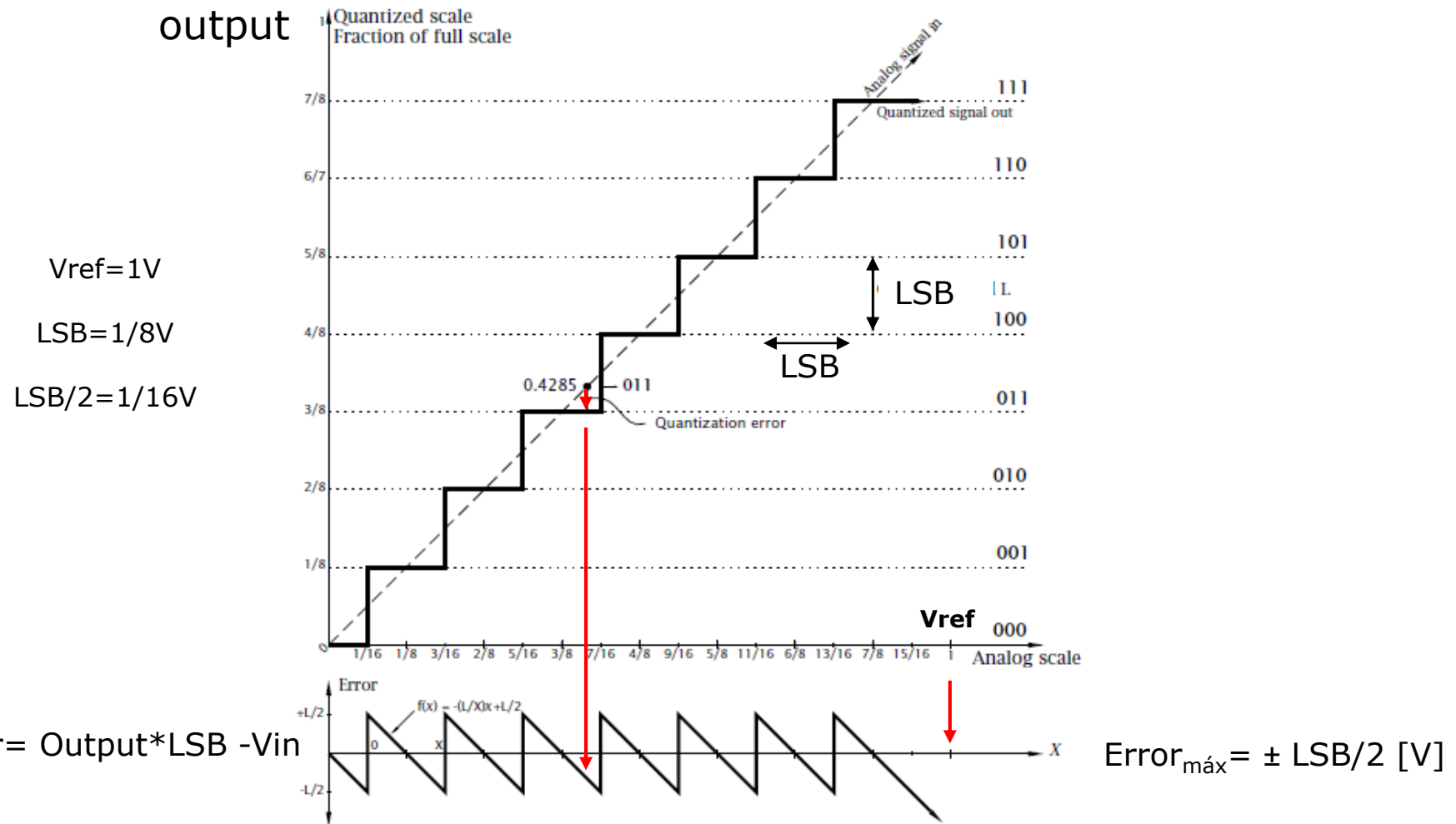
$$V_{in} = RefN + Output \cdot LSB$$

Cuando V_{in} coincide con la mitad de LSB

$$\begin{aligned} \text{Rango Fondo de Escala} \\ FSR = (RefP - RefN) - LSB \end{aligned}$$

Introducción al periférico ADC

- Transferencia ideal Entrada-Salida de un ADC de 3bits y el error de cuantización



Introducción al periférico ADC

- **Resolución o Paso de cuantización:**

Es el mínimo cambio en la entrada analógica que produce un cambio en el código de salida de un LSB

$$\text{LSB [V]} = V_{\text{ref}} / 2^N \quad \text{siendo N: \# de bits}$$

- Ejemplos:

Table 13-1: Resolution versus Step Size for ADC ($V_{\text{ref}} = 5 \text{ V}$)

<i>n</i> -bit	Number of steps	Step size (mV)
8	256	$5/256 = 19.53$
10	1024	$5/1024 = 4.88$
12	4096	$5/4096 = 1.2$
16	65,536	$5/65,536 = 0.076$

Notes: $V_{CC} = 5 \text{ V}$

Step size (resolution) is the smallest change that can be discerned by an ADC.

- **Error de cuantización:**

$$\text{Error}_{\text{máx}} = \pm \text{LSB}/2 \text{ [V]}$$

Otros errores en un ADC real

Resumiendo:

- En la implementación de los ADC existen diversos errores que modifican la transferencia entrada-salida ideal.
 - El fabricante especifica entonces un cota de error absoluto garantizada para sus dispositivos fabricados sin ajuste en fábrica.
 - Podemos ver la definición y el valor en la hoja de datos del dispositivo:
- **Absolute Accuracy:** The maximum deviation of an actual (unadjusted) transition compared to an ideal transition for any code. This is the compound effect of Offset, Gain Error, Differential Error, Non-linearity, and Quantization Error. Ideal value: ± 0.5 LSB.
- Existen técnicas de calibración y compensación para minimizar los errores de implementación. El límite de esta mejora es el error de cuantización.

Errores en un ADC real

- En aplicaciones de audio, video o comunicaciones por ejemplo, los errores de un ADC se cuantifican con otros parámetros a saber:
 - Bajo estrictas suposiciones, el error de cuantización se puede considerar como un proceso aleatorio llamado *ruido de cuantización*. Esto permite cuantificar la calidad del ADC a través de la relación señal a ruido *SNR*:

$$SNR(dB) = 20 \log \left(\frac{V_{RMS(Signal)}}{V_{RMS(Noise)}} \right) \quad SNR(dB) = 6.02N + 1.76(dB)$$

- Otro parámetro de interés es la distorsión armónica total *THD* que cuantifica las no linealidades del ADC midiendo la relación entre las amplitudes de las armónicas producidas y la amplitud de un tono puro original:

$$THD = \frac{\sqrt{V_2^2 + V_3^2 + \dots + V_n^2}}{V_1}$$

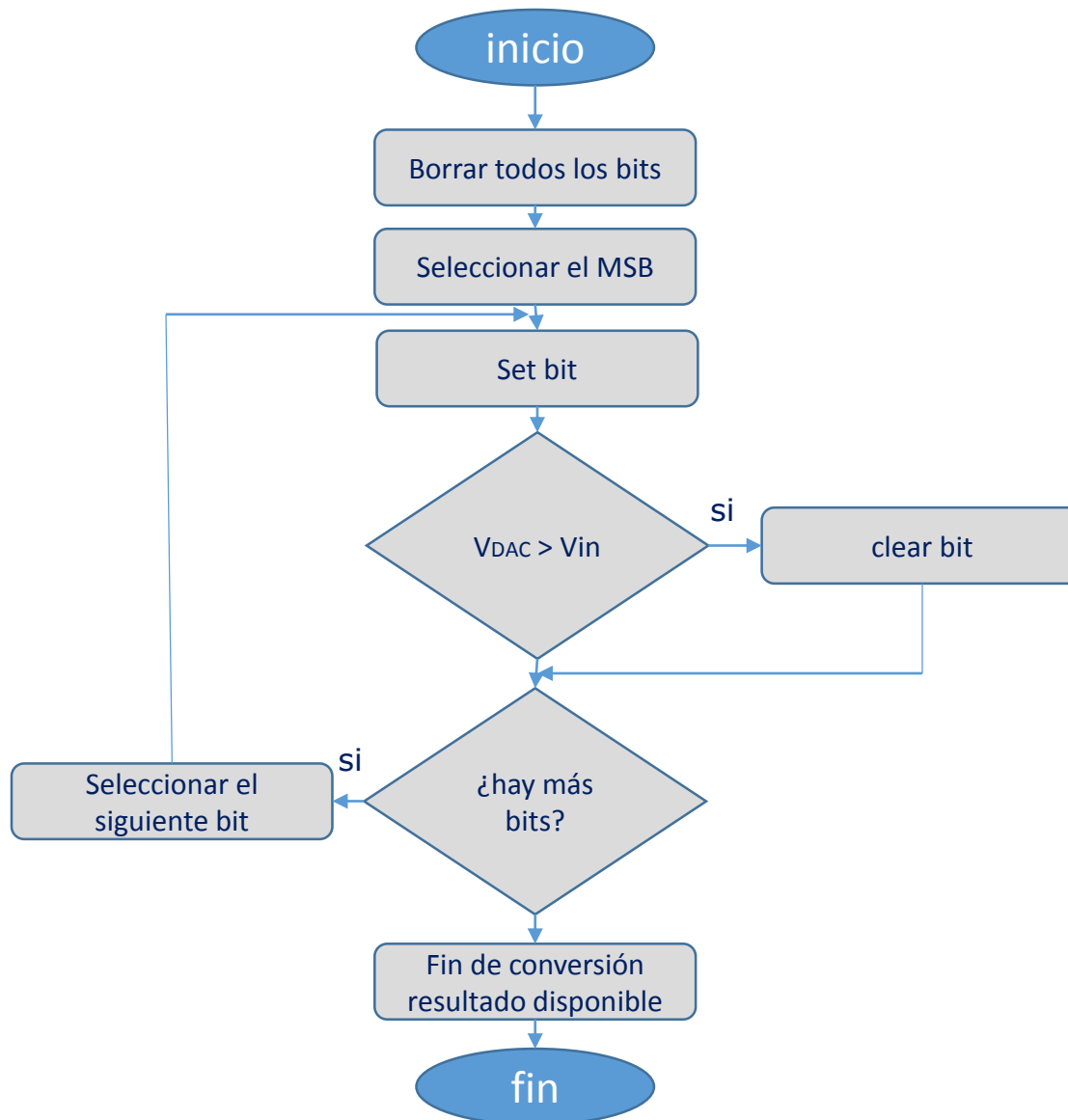
- La combinación de los dos parámetros anteriores definen el comportamiento dinámico del conversor y se denomina *SINAD* (relación señal a ruido+distorsión):

$$SINAD = -10 \log(10^{-SNR/10} + 10^{-THD/10})$$

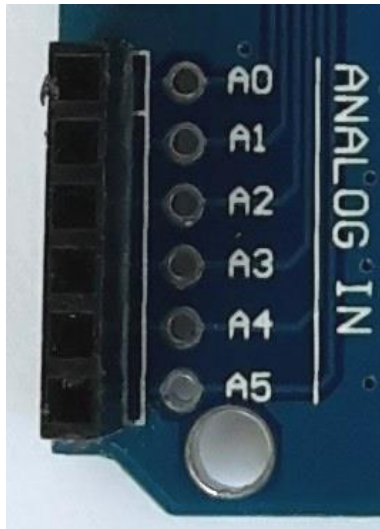
- Finalmente, los errores cuantificables del ADC permiten definir un número efectivo de bits para su uso *ENOB* (Effective number of bits):

$$ENOB = \frac{SINAD - 1.76}{6.02}$$

ADC: Algoritmo de Aproximaciones Sucesivas



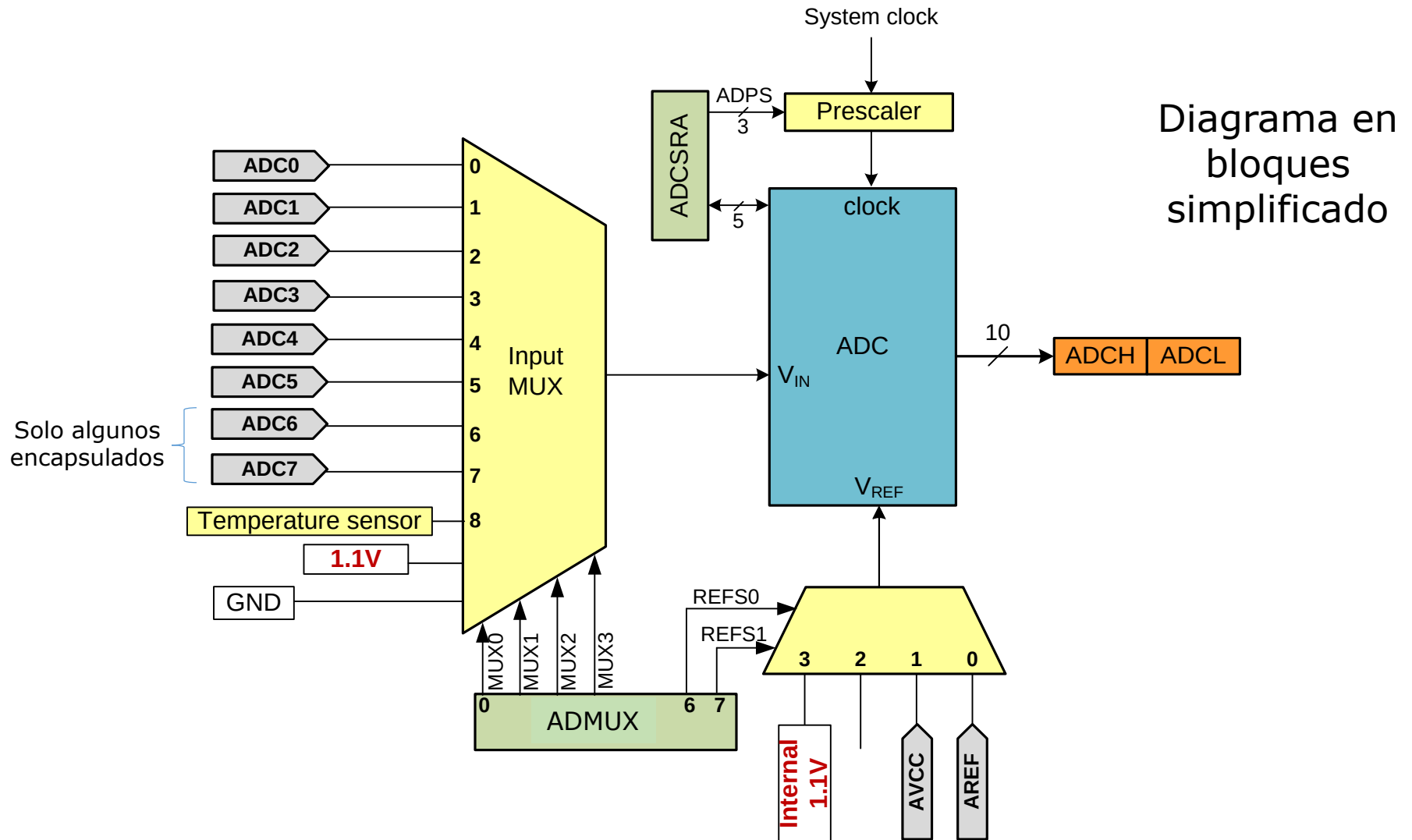
Periférico ADC en Atmega AVR



		28 pin	
(PCINT14/ <u>RESET</u>) PC6	1	28	PC5 (ADC5 /SCL/PCINT13)
(PCINT16/RXD) PD0	2	27	PC4 (ADC4 /SDA/PCINT12)
(PCINT17/TXD) PD1	3	26	PC3 (ADC3 /PCINT11)
(PCINT18/INT0) PD2	4	25	PC2 (ADC2 /PCINT10)
(PCINT19/OC2B/INT1) PD3	5	24	PC1 (ADC1 /PCINT9)
(PCINT20/XCK/T0) PD4	6	23	PC0 (ADC0 /PCINT8)
VCC	7	22	GND
GND	8	21	AREF
(PCINT6/XTAL1/TOSC1) PB6	9	20	AVCC
(PCINT7/XTAL2/TOSC2) PB7	10	19	PB5 (SCK/PCINT5)
(PCINT21/OC0B) PD5	11	18	PB4 (MISO/PCINT4)
(PCINT22/OC0A/AIN0) PD6	12	17	PB3 (MOSI/OC2A/PCINT3)
(PCINT23/AIN1) PD7	13	16	PB2 (SS/OC1B/PCINT2)
(PCINT0/CLKO/ICP1) PB0	14	15	PB1 (OC1A/PCINT1)

MEGA328

ADC del Atmega328



Registros de Programación

- ADMUX

ADMUX – ADC Multiplexer Selection Register

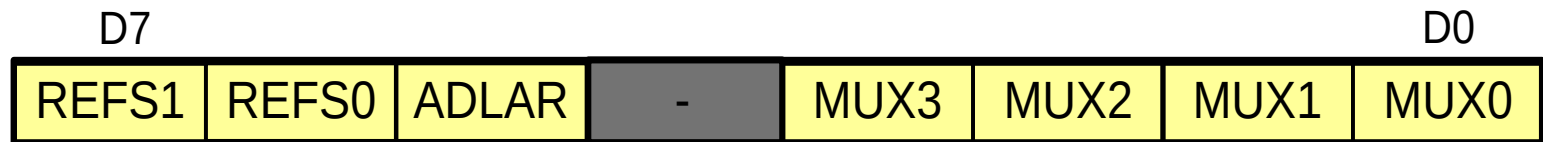
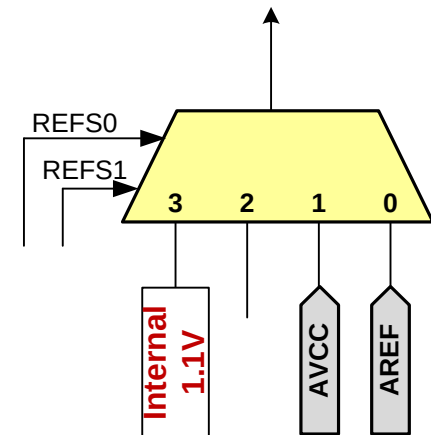


Table 13-4: V_{ref} Source Selection Table for AVR

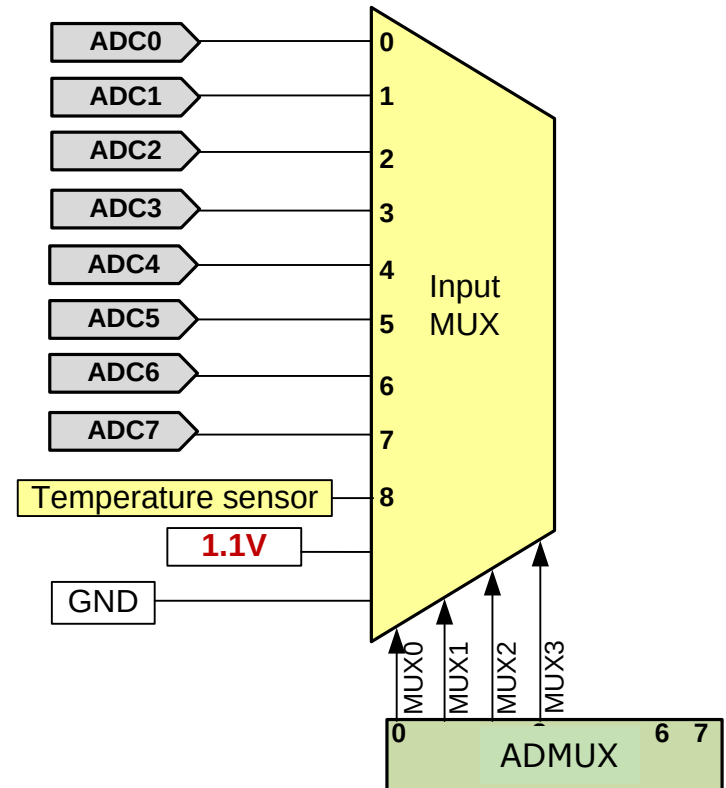
REFS1	REFS0	V_{ref}	
0	0	AREF pin	Set externally
0	1	AVCC pin	Same as VCC
1	0	Reserved	----
1	1	Internal 1.1V	Fixed regardless of VCC value



Registros de Programación

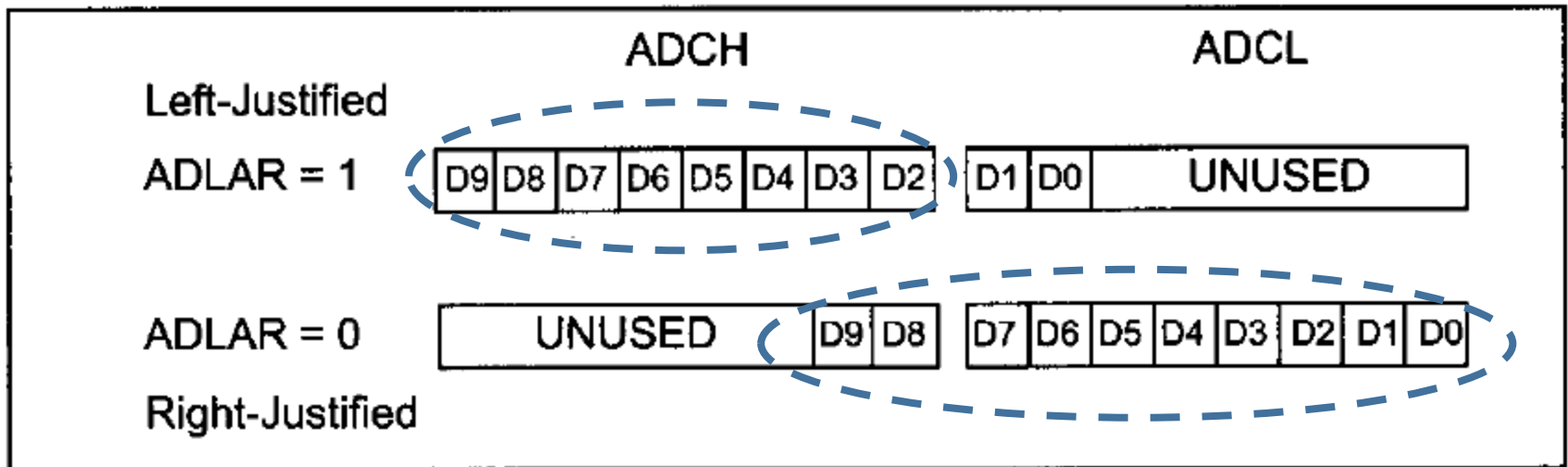
- Selección de Canal

MUX3...0	Single-ended Input
00000	ADC0
00001	ADC1
00010	ADC2
00011	ADC3
00100	ADC4
00101	ADC5
00110	ADC6
00111	ADC7



Registros de Programación

- Justificación (ADLAR):



```
uint16_t result =ADC; //resultado completo de 10 bits
```

```
uint8_t result =ADCH; //resultado de 8 bits justificado a izq.
```

$$V_{\text{medido}} = (\text{result} * V_{\text{ref}}) / 2^N$$

Registros de Programación

- ADCSRA

ADEN	ADSC	ADATE	ADIF	ADIE	ADPS2	ADPS1	ADPS0
------	------	-------	------	------	-------	-------	-------

ADEN- Bit7 ADC Enable

This bit enables or disables the ADC. Writing this bit to one will enable and writing this bit to zero will disable the ADC even while a conversion is in progress.

ADSC- Bit6 ADC Start Conversion

To start each conversion you have to write this bit to one.

ADATE- Bit5 ADC Auto Trigger Enable

Auto Triggering of the ADC is enabled when you write this bit to one.

ADIF- Bit4 ADC Interrupt Flag

This bit is set when an ADC conversion completes and the Data Registers are updated

ADIE- Bit3 ADC Interrupt Enable

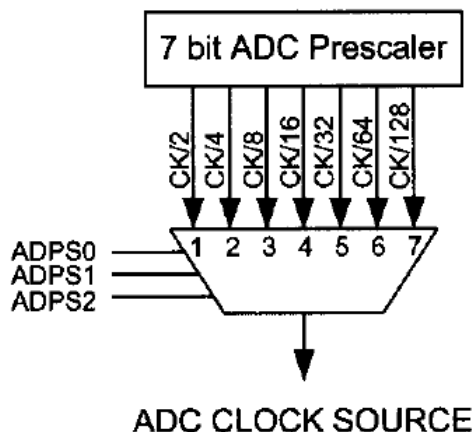
Writing this bit to one enables the ADC Conversion Complete Interrupt.

ADPS2:0- Bit2:0 ADC Prescaler Select Bits

These bits determine the division factor between the XTAL frequency and the input clock to the ADC.

Registros de Programación

- ADPS bits
(Prescalador del reloj ADC)



ADPS2	ADPS1	ADPS0	ADC Clock
0	0	0	Reserved
0	0	1	CK/2
0	1	0	CK/4
0	1	1	CK/8
1	0	0	CK/16
1	0	1	CK/32
1	1	0	CK/64
1	1	1	CK/128

- Notar que se garantiza la mejor performance hasta un ADCLK de 200KHz pero el máx ADCLK es 1MHz.
 - Ej.: para 8MHz de reloj CPU, el pre-divisor debe ser 64 o 128 tal que $8\text{MHz}/64=125\text{KHz}$ ($<200\text{kHz}$)
- ADSC Start conversion:
 - Eligiendo ADIF como trigger
Se obtiene muestreo continuo o Free running.
(ver tabla 23.6 del data sheet)

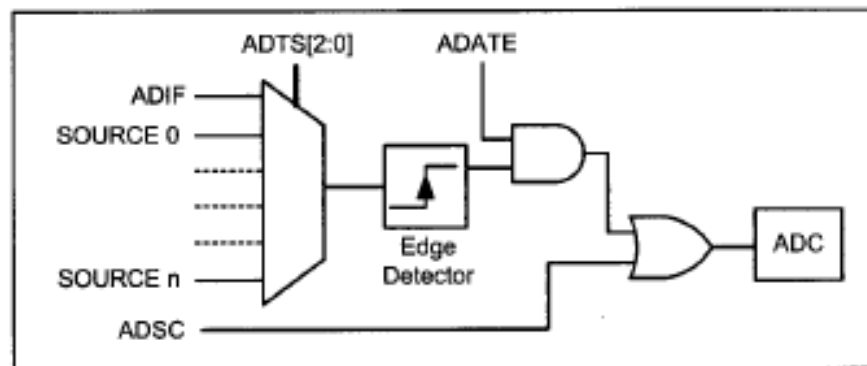


Figure 13-11. AVR ADC Trigger Source

Registros de Programación

- Tiempos de conversión

Figure 23-5. ADC Timing Diagram, Single Conversion

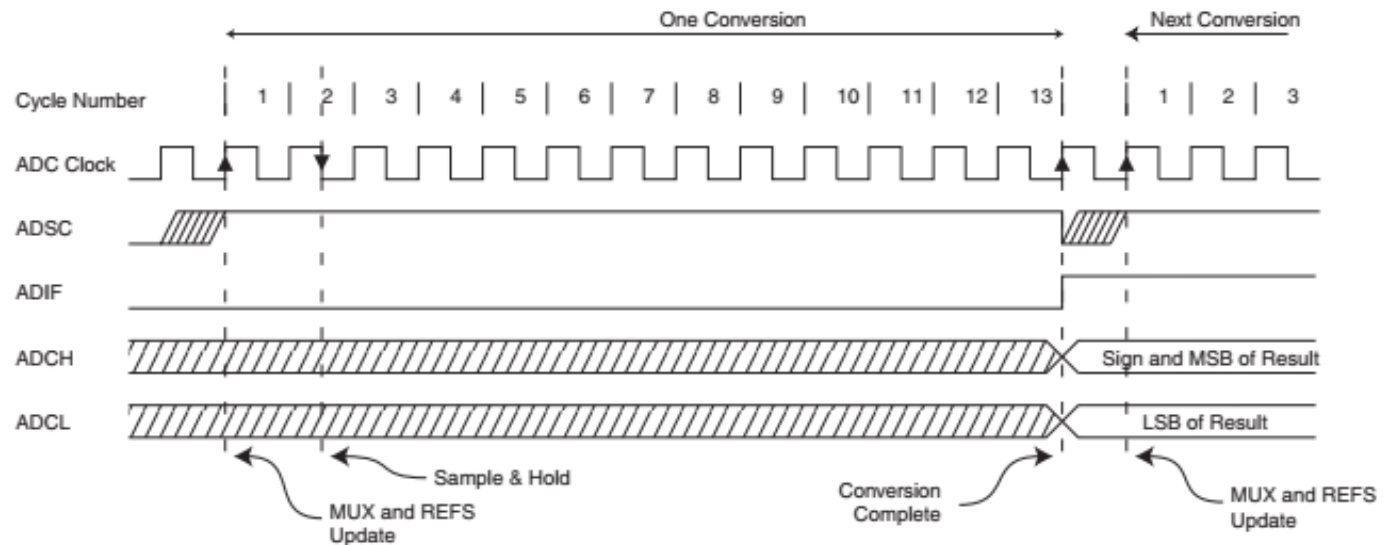


Table 23-1. ADC Conversion Time

Condition	Sample & Hold (Cycles from Start of Conversion)	Conversion Time (Cycles)
First conversion	13.5	25
Normal conversions, single ended	1.5	13
Auto Triggered conversions	2	13.5

*Ejemplo: para ADCLK= 125KHz, en conversión normal tenemos un tiempo de conversión de 104us
Para este caso la máx tasa de muestreo sería 9.6KHz y el ancho de banda de señal 4.8KHz ($f_{max}/2$)*

Programación ADC por polling

1. Configurar como entrada el terminal del canal seleccionado(DIDR0 register)
2. Encender el ADC
3. Configurar el reloj y tipo de conversión
4. Seleccionar el canal y la tensión de referencia Vref
5. Iniciar la conversión escribiendo un 1 en el bit ADSC del ADCSRA.
6. Esperar a que la conversión finalice encuestando el bit ADIF del ADCSRA.
7. Una vez que ADIF =1, leer el resultado en el ADC register.
8. Borrar flag escribiendo 1 en ADIF.
9. Si desea volver a muestrear el mismo canal ir al paso 5.
10. Si desea seleccionar otro canal u otra Vref ir al paso 4.

Programación ADC por polling

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>
```

- Este programa toma una muestra del ch0 (ADC0) y muestra el resultado en el Port B y Port D.

```
int main (void)
{
    DDRB = 0xFF; //make Port B an output
    DDRD = 0xFF; //make Port D an output
    DIDR0 = 0x01; //Digital Input Disable (opcional)
    ADCSRA = 0x87; //make ADC enable and select ck/128
    ADMUX = 0x00; // Vref=AVCC, right-justified, ADC0 pin

    while(1)
    {
        ADCSRA |= (1<<ADSC); //start conversion

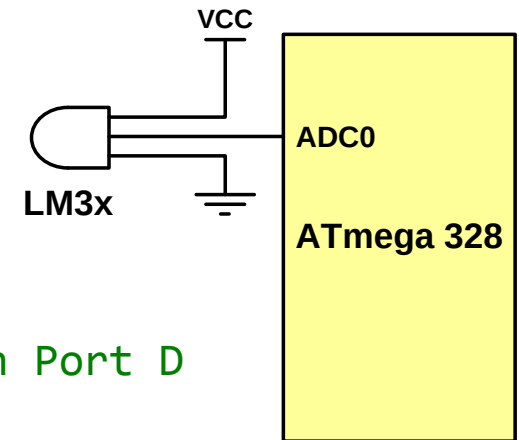
        while((ADCSRA & (1<<ADIF)) == 0); //wait for conversion to finish

        ADCSRA |= (1<<ADIF); //borrar flag

        PORTD = ADCL; //give the low byte to PORTD
        PORTB = ADCH; //give the high byte to PORTB
        _delay_ms(100);
    }
}
```

Nota: HAY QUE BORRAR EL FLAG ADIF SI ES POR POLLING Y SE BORRA AUTOMATICAMENTE SI ES POR INTERRUPCION

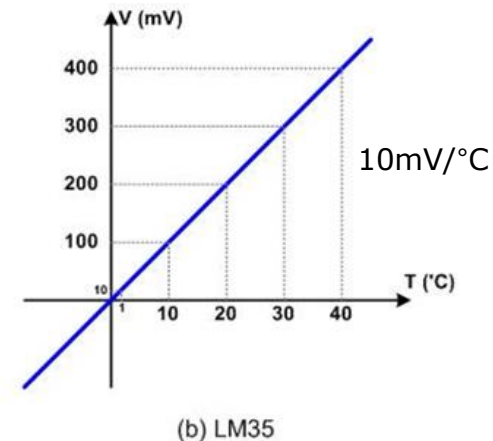
Usando el LM35



//this program reads the sensor and displays it on Port D

```
int main (void)
{
    DDRB = 0xFF; //make Port B an output
    DDRC = 0;     //make Port C an input for ADC input
    ADCSRA = 0x87; //make ADC enable and select ck/128
    ADMUX = 0xC0; //1.1V Vref, ADC0, right-justified

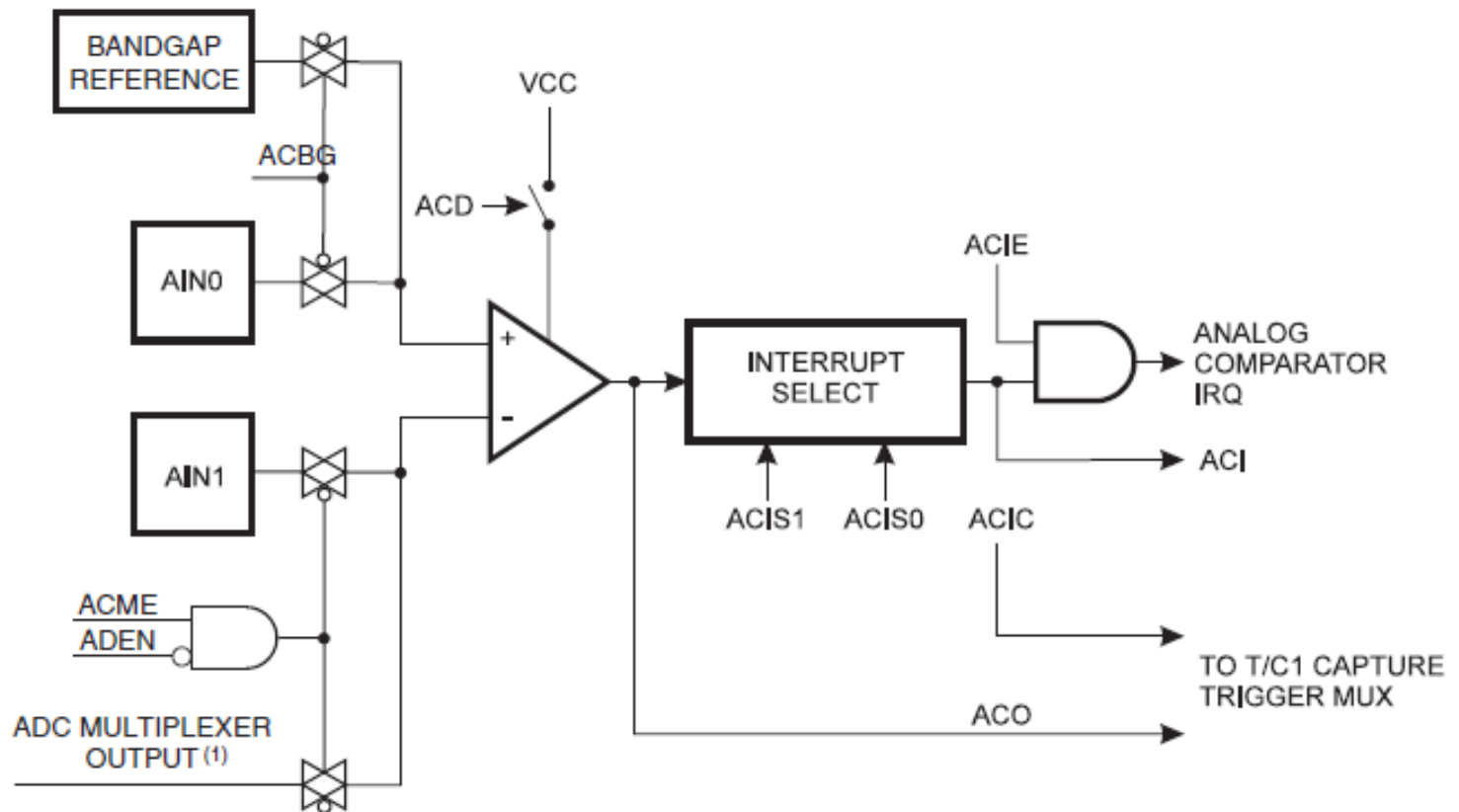
    while (1){
        ADCSRA |= (1<<ADSC); //start conversion
        while((ADCSRA & (1<<ADIF)) == 0); //wait for end of conversion
        ADCSRA |= (1<<ADIF); //clear the ADIF flag
        PORTB = (ADCW*10)/93; //PORTB = adc value/9.3
        _delay_ms(1000);
    }
}
```



Comparador Analógico

- ADC de 1bit

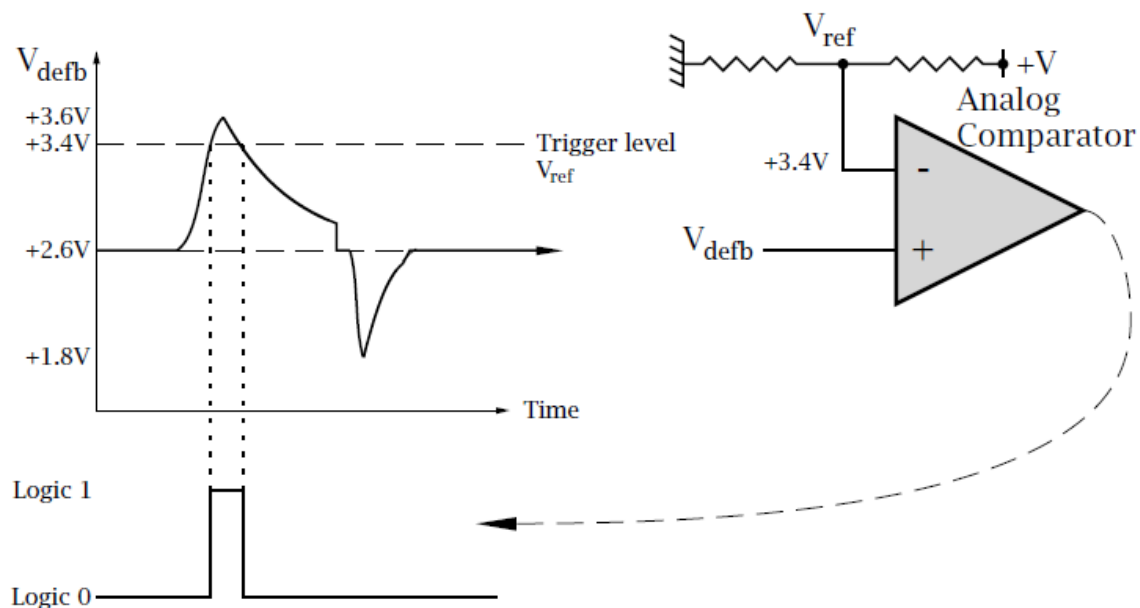
cáp. 23 (hoja de datos)



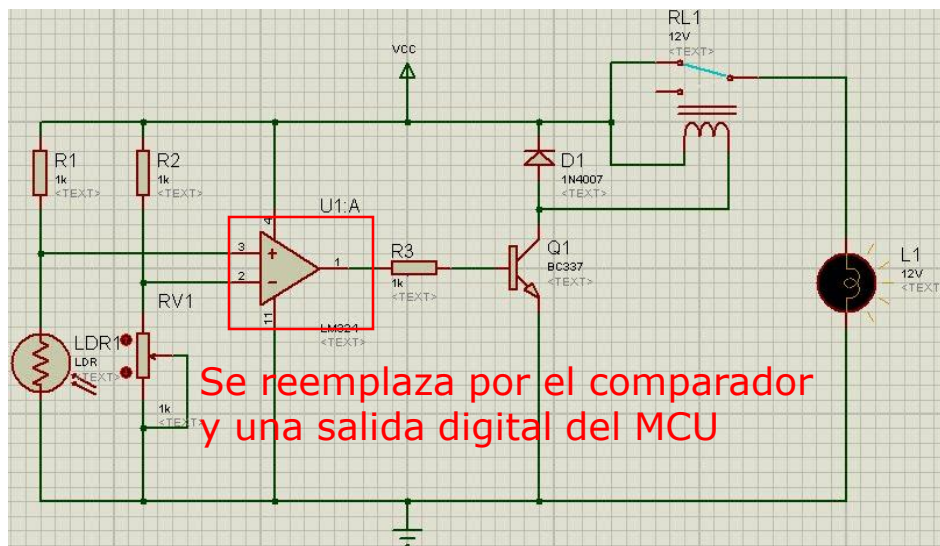
$ACO = 1$ si $AIN0 > AIN1$, $ACO=0$ caso contrario

Ejemplos de uso:

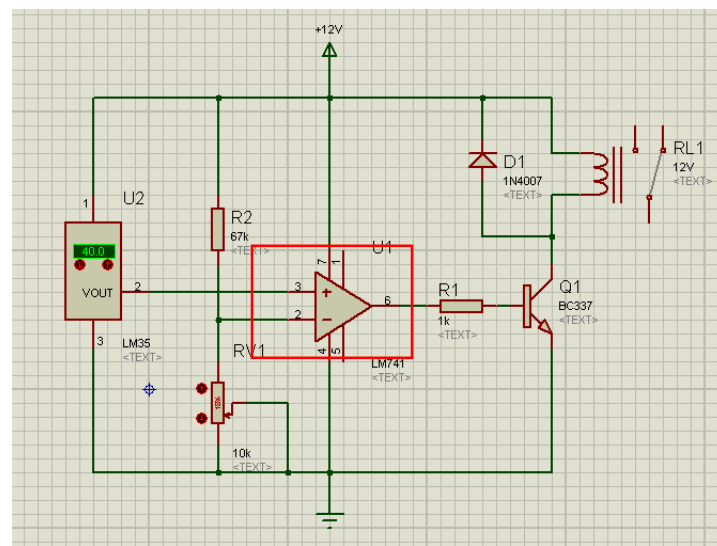
- Detector de sobretensión



- Ejemplo detector de luz



- Ejemplo: alarma de temperatura



Ver códigos en ejemplos 5.3 y 5.4 del libro AVR de Espinoza